

CS-XXX: Introduction to Machine Learning

Linear Regression

Arbish Akram

Department of Computer Science
Government College University

Slides by Dr. Nazar Khan

Notation

- ▶ Scalars are denoted by lower-case letters like s, a, b .
- ▶ Vectors are denoted by lower-case bold letters like $\mathbf{x}, \mathbf{y}, \mathbf{v}$.
- ▶ Matrices are denoted by upper-case bold letters like $\mathbf{M}, \mathbf{D}, \mathbf{A}$.
- ▶ Any vector $\mathbf{x} \in \mathbb{R}^d$ is by default a column vector.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

- ▶ The corresponding row vector is obtained as $\mathbf{x}^T = [x_1 \quad x_2 \quad \dots \quad x_d]$.

Inner Product

For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$

- ▶ *Inner product* is a scalar value.

$$\mathbf{x}^T \mathbf{y} = x_1 y_1 + x_2 y_2 + \cdots + x_d y_d = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$$

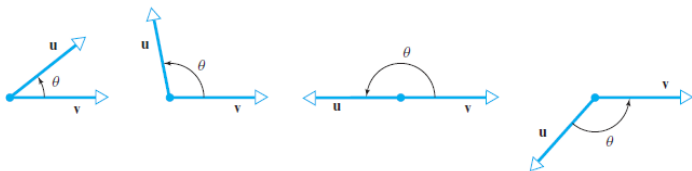
where θ is the angle between vectors $\mathbf{x}$ and $\mathbf{y}$.

- ▶ Also called *dot product* or *scalar product*. Other representations:

$$\mathbf{x} \cdot \mathbf{y}, (\mathbf{x}, \mathbf{y}) \text{ and } \langle \mathbf{x}, \mathbf{y} \rangle$$

- ▶ Represents similarity of vectors.

- ▶ If $\mathbf{x}^T \mathbf{y} = 0$, then $\mathbf{x}$ and $\mathbf{y}$ are orthogonal vectors (in 2D, this means they are perpendicular).



Euclidean Norm

- *Euclidean norm* of vector

$$\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}} = \sqrt{x_1 x_1 + x_2 x_2 + \cdots + x_d x_d}$$

represents the magnitude of the vector.

- *Euclidean distance* between points $\mathbf{x}$ and $\mathbf{y}$ can be computed as

$$\begin{aligned}\|\mathbf{x} - \mathbf{y}\| &= \sqrt{(\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y})} \\ &= \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_d - y_d)^2}\end{aligned}$$

- *Unit vector* has norm 1. Also called *normalised vector*.
- If $\|\mathbf{x}\| = 1$ and $\|\mathbf{y}\| = 1$, and $\mathbf{x}^T \mathbf{y} = 0$, then $\mathbf{x}$ and $\mathbf{y}$ are *orthonormal vectors*.

Outer Product

For vectors $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{z} \in \mathbb{R}^k$

- *Outer-product* $\mathbf{x}\mathbf{z}^T$ is a $d \times k$ matrix.

$$\mathbf{x}\mathbf{z}^T = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \begin{bmatrix} z_1 & z_2 & \dots & z_k \end{bmatrix} = \begin{bmatrix} x_1 z_1 & x_1 z_2 & \dots & x_1 z_k \\ x_2 z_1 & x_2 z_2 & \dots & x_2 z_k \\ \vdots & \vdots & \vdots & \vdots \\ x_d z_1 & x_d z_2 & \dots & x_d z_k \end{bmatrix}$$

Matrix and Vector Calculus

For vector $\mathbf{x} \in \mathbb{R}^d$, scalar function $f(\mathbf{x})$ and vector function $\mathbf{g}(\mathbf{x}) \in \mathbb{R}^k$

- ▶ The gradient operator $\frac{d}{d\mathbf{x}}$ is also written as $\nabla_{\mathbf{x}}$ or simply ∇ when the differentiation variable is implied.

$$\text{▶ } \nabla_{\mathbf{x}} = \begin{bmatrix} \frac{\partial}{\partial x_1} \\ \frac{\partial}{\partial x_2} \\ \vdots \\ \frac{\partial}{\partial x_d} \end{bmatrix} \text{ so that } \nabla_{\mathbf{x}}(f(\mathbf{x})) = \frac{d}{d\mathbf{x}}(f(\mathbf{x})) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix}$$

$$\text{▶ } \nabla_{\mathbf{x}}(\mathbf{g}(\mathbf{x})) = \frac{d}{d\mathbf{x}}(\mathbf{g}(\mathbf{x})) = \begin{bmatrix} \frac{\partial g_1(\mathbf{x})}{\partial x_1} & \frac{\partial g_2(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial g_k(\mathbf{x})}{\partial x_1} \\ \frac{\partial g_1(\mathbf{x})}{\partial x_2} & \frac{\partial g_2(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial g_k(\mathbf{x})}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial g_1(\mathbf{x})}{\partial x_d} & \frac{\partial g_2(\mathbf{x})}{\partial x_d} & \cdots & \frac{\partial g_k(\mathbf{x})}{\partial x_d} \end{bmatrix}$$

Regression

- ▶ We study the problem of *regression*.
 - ▶ Predict *continuous* target variable(s) t given input variables vector x .
- ▶ Given training data $\{(x_1, t_1), \dots, (x_N, t_N)\}$, learn a function $y(x, w)$ that maps the inputs to the targets.
- ▶ Regression corresponds to finding the optimal parameters w^* .

Linear Regression

- ▶ The simplest regression model is *linear regression*.
- ▶ Linear in parameters w and linear in inputs x .

$$y(x, w) = w^T x = w_0 + w_1 x_1 + \cdots + w_D x_D$$

- ▶ Parameter w_0 accounts for a fixed offset in the data and is called the *bias* parameter.
- ▶ To incorporate bias, we have increased the dimensionality of x from D to $D + 1$ by appending a 1 before it.
- ▶ This makes our input vector $x \in \mathbb{R}^{D+1}$ and parameter vector $w \in \mathbb{R}^{D+1}$.

Linear Regression

- ▶ Linear models are significantly limited for practical problems – especially for high dimensional inputs.
- ▶ However, they have nice analytical properties and they form the foundation for more sophisticated machine learning approaches.

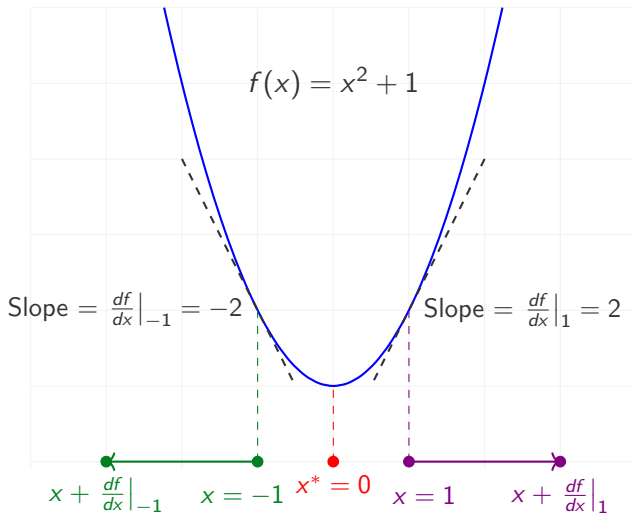
Linear Regression

- ▶ A more powerful model is linear in parameters w but non-linear in inputs x .

$$y(x, w) = w^T \phi(x) = w_0 \phi_0(x) + w_1 \phi_1(x) + \cdots + w_M \phi_M(x)$$

- ▶ $\phi_0(x)$ is usually set to 1 to make w_0 the bias parameter.
- ▶ Note that now $w \in \mathbb{R}^{M+1}$ where M is not necessarily equal to D .
- ▶ The input x -space is non-linearly mapped to ϕ -space and learning takes place in this new ϕ -space.
- ▶ While the learning remains linear, the learned mapping is actually non-linear in x -space.

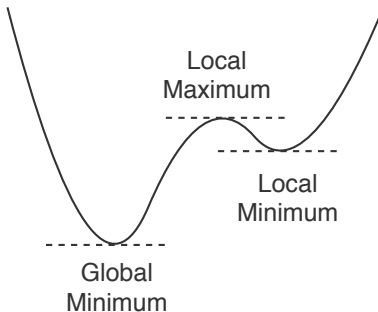
Minimization



What is the slope/derivative/gradient at the minimizer $x^* = 0$?

Minimization

Local vs. Global Minima



- ▶ *Stationary point*: where derivative is 0.
- ▶ A stationary point can be a minimum or a maximum.
- ▶ A minimum can be local or global. Same for maximum.

Linear Regression

- ▶ Error function of a regression model is

$$E(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^T \phi(x_n)\}^2$$

- ▶ Derivative with respect to w is

$$\frac{d}{dw} E(w) = \sum_{n=1}^N \{t_n - w^T \phi(x_n)\} \phi(x_n)^T$$

- ▶ At the minimiser w^* , the gradient must be equal to 0

$$\left. \frac{d}{dw} E(w) \right|_{w^*} = 0$$

Linear Regression

- Equating gradient to the 0 vector

$$\sum_{n=1}^N t_n \phi(x_n)^T - w^{*T} \left(\sum_{n=1}^N \phi(x_n) \phi(x_n)^T \right) = 0 \quad (1)$$
$$\Rightarrow w^{*T} = \left(\sum_{n=1}^N t_n \phi(x_n)^T \right) \left(\sum_{n=1}^N \phi(x_n) \phi(x_n)^T \right)^{-1}$$

Linear Regression

- ▶ To convert to a pure matrix-vector notation without summations, let us define the following $N \times M$ matrix

$$\Phi = \begin{bmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_{M-1}(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_{M-1}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_{M-1}(x_N) \end{bmatrix}$$

known as the *design matrix*.

Linear Regression

- ▶ It can be verified that the second term in Equation (1) $\sum_{n=1}^N \phi(x_n)\phi(x_n)^T = \Phi^T \Phi$. (Verify this.)
- ▶ By placing the target values in a vector $\mathbf{t} = (t_1, \dots, t_N)^T$ we can also write the first term as $\Phi^T \mathbf{t}$. (Verify this.)
- ▶ Now we can solve for the optimal weights as

$$\mathbf{w}^* = \underbrace{(\Phi^T \Phi)^{-1} \Phi^T}_{\Phi^\dagger} \mathbf{t}$$

- ▶ The $M \times N$ matrix $\Phi^\dagger$ is known as the *Moore-Penrose pseudo-inverse* or simply *pseudo-inverse* of matrix Φ .
- ▶ It is a generalisation of matrix inverse to non-square matrices.
- ▶ For a square, invertible matrix Φ , it can be verified that $\Phi^\dagger = \Phi^{-1}$. (Verify this.)

Linear Regression

Regularisation

- ▶ Error function for regularised linear regression is

$$E(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^T \phi(x_n)\}^2 + \frac{\lambda}{2} \|w\|^2$$

where λ is the *regularisation coefficient* that controls the trade-off between fitting and regularisation.

- ▶ This is also known as *regularised least squares*.
- ▶ Such regularisation is also called *weight decay* or *parameter shrinkage* because it encourages weight/parameter values to remain close to 0.
- ▶ Regularisation allows more complex models to be trained on small datasets without severe over-fitting.
- ▶ However, parameter λ needs to be set appropriately.

Linear Regression

Regularised

- ▶ Optimal solution to regularised linear regression is

$$\mathbf{w}^* = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

Linear Regression

Multivariate targets

- ▶ For the case of multivariate target vectors $\mathbf{t}_n \in \mathbb{R}^K$, we are interested in the multivariate mapping $y(\mathbf{x}, \mathbf{W}) = \mathbf{W}^T \Phi(\mathbf{x})$.
- ▶ Column k of the $M \times K$ matrix $\mathbf{W}$ determines the mapping from $\phi(\mathbf{x})$ to the k_{th} output component.
- ▶ The optimal solution given training data $\{\mathbf{x}_n, \mathbf{t}_n\}_{n=1}^N$ can be computed as

$$\mathbf{W}^* = \Phi^\dagger \mathbf{T}$$

where $\mathbf{T} = \begin{bmatrix} \mathbf{t}_1^T \\ \vdots \\ \mathbf{t}_N^T \end{bmatrix}$ is the $N \times K$ matrix of target vectors.